

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF
ENGINEERING)** Department of Computer Science and
Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06 12	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	M.S.Dinesh	Reg. No.:	192424016
Section / Batch:	I	Date:	22.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Code of Conduct

I (M.S.Dinesh / 192424016) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___ M.S.Dinesh ___

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				

Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1 Maximum Product Subarray

5 marks

Given an array of integers, find the contiguous subarray that has the maximum product. The array may contain negative numbers, which can affect the product value. The goal is to determine the maximum possible product. Use Dynamic Programming to compute the result efficiently. Input Format: n = number of elements arr[] = array elements
Sample Input: n = 5 arr = 2 3 -2 4 -1 Expected Output: Max Product = 48

Solution

Introduction

- **Core Goal:** Finds the contiguous sequence of elements within a one-dimensional array that yields the highest multiplication result.
- **Negative Impact:** Unlike the maximum sum problem, multiplying by a negative number can turn a very small minimum product into a massive maximum product.
- **Zero Handling:** Encountering a zero completely resets the product sequence, requiring the algorithm to effectively restart its tracking from the next position.
- **Dynamic Essence:** Evaluates every subarray combination efficiently without recalculating the products of overlapping segments from scratch.
- **Practical Use:** Applies heavily to financial portfolio analysis, genomic sequence analysis,

and pattern recognition in data streams.

Problem Solution (Python)

This optimal implementation uses Dynamic Programming (Kadane's variation) to solve the problem in $O(n)$ time and $O(1)$ space.

```
def maxProduct(arr):
    if not arr:
        return 0

    global_max = arr[0]
    curr_max = arr[0]
    curr_min = arr[0]

    for i in range(1, len(arr)):
        num = arr[i]

        # If the number is negative, max and min swap roles when multiplied
        if num < 0:
            curr_max, curr_min = curr_min, curr_max

        # 5 Key solution steps map here:
        curr_max = max(num, curr_max * num)
        curr_min = min(num, curr_min * num)
        global_max = max(global_max, curr_max)

    return global_max

# Example execution
arr = [2, 3, -2, 4, -1]
print(f"Max Product = {maxProduct(arr)}") # Output: 48
```

Algorithm

- **Initialize States:** Store the first element as the global maximum, current maximum, and current minimum variables.
- **Iterate Array:** Loop through the integer array sequentially starting from the second element (index 1) to the end.
- **Handle Negatives:** Swap the current maximum and current minimum values whenever a negative number is encountered.
- **Update Local Bounds:** Compute the new current maximum and minimum by comparing the current element alone against its product with previous bounds.
- **Update Global Result:** Compare and update the global maximum with the newly calculated current local maximum at each step.

Formulae

At each index i , the dynamic programming transitions depend on these mathematical choices:

- **Current Maximum:** $dp_{max}[i] = \max(\vec{arr}[i], \max(\vec{dp_{max}}[i-1] \cdot \vec{arr}[i], \vec{dp_{min}}[i-1] \cdot \vec{arr}[i]))$
- **Current Minimum:** $dp_{min}[i] = \min(\vec{arr}[i], \min(\vec{dp_{max}}[i-1] \cdot \vec{arr}[i], \vec{dp_{min}}[i-1] \cdot \vec{arr}[i]))$
- **Global Output:** $Result = \max_{0 \leq i < n}(dp_{max}[i])$
- **Time Complexity:** $O(n)$ operations because the array is traversed exactly once.
- **Space Complexity:** $O(1)$ storage since only three tracking variables are maintained during execution.

Advantages

- **Linear Efficiency:** Runs in optimal linear time, making it exceptionally fast even for arrays containing millions of integers.
- **Minimal Memory:** Requires constant auxiliary space, avoiding overhead and saving system RAM during heavy processing.
- **Single Pass:** Processes the data sequentially in one directional sweep without needing to look back or re-read inputs.
- **Robust Tracking:** Handles complex combinations of positive numbers, negative numbers, and zeroes flawlessly without separate conditional branches.
- **Online Adaptability:** Fits streaming data environments where elements arrive one by one and results must update continuously.

Limitations

- **Integer Overflow:** Multiplying many integers continuously can quickly exceed standard

32-bit or 64-bit integer limits, causing storage overflows.

- **Index Disconnect:** Finds the peak product value successfully but does not inherently track the starting and ending indices of the subarray.
- **Contiguity Constraint:** Cannot solve problems where elements in the subset are allowed to be non-consecutive or scattered.
- **Substructure Sensitivity:** Small modifications to array values require a complete recalculation rather than a localized patch update.

Output

 **SIMATS**
ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

DINESH M S
192424016RC

CO4 AT1-CODE CHALLENGE TASK

← Back

Language: Python C C++ Java

QUESTIONS**Q1**
Maximum Product Subarray
5 marks
1/1 answered**Q1 Maximum Product Subarray** 5 marks

Given an array of integers, find the contiguous subarray that has the maximum product. The array may contain negative numbers, which can affect the product value. The goal is to determine the maximum possible product. Use Dynamic Programming to compute the result efficiently. Input Format: n = number of elements arr[] = array elements Sample Input: n = 5 arr = 2 3 -2 4 -1 Expected Output: Max Product = 48

Your Code**Input**
5
2 3
-2 4
-1**Output**
Max Product = 48